

UNIVERSIDADE DO OESTE DE SANTA CATARINA

ERIC CAMINI
MAIARA BRAUN KOTHE
MATHEUS SCHERER BAMBERG
MOISÉS ZILLES

VESTOCK

DISCIPLINA: PROGRAMAÇÃO III
PROFESSOR: WAGNER TITON

SÃO MIGUEL DO OESTE - SC
2026

1. INTRODUÇÃO

Este trabalho acadêmico tem como objetivo geral desenvolver uma solução web completa e funcional utilizando um framework moderno de desenvolvimento. A aplicação prática visa consolidar os conceitos de arquitetura de software discutidos em aula. Para fundamentar a engenharia do sistema, estabeleceu-se como meta obrigatória a implementação de uma arquitetura baseada em divisão clara de papéis, como a Arquitetura em Camadas ou o modelo MVC, além da integração de, no mínimo, três padrões de projeto distintos para mitigar problemas recorrentes de acoplamento e persistência.

Como proposta prática para o cumprimento dessas diretrizes, o grupo idealizou o desenvolvimento do Sistema Vestock. Trata-se de uma plataforma web voltada especificamente para o cadastro de vendas e o controle de estoque de uma loja de roupas. Com um escopo restrito e direcionado, o sistema é de uso exclusivo dos funcionários do estabelecimento, operando como uma ferramenta corporativa interna para otimizar o fluxo operacional, mitigar inconsistências no inventário e registrar transações comerciais em tempo real.

Para viabilizar o ecossistema do Sistema Vestock, o projeto compreende estruturalmente um *front-end* web responsivo desenvolvido em Flutter e Dart, focado na experiência do usuário interno, e um *back-end* estruturado em Java 21 com Spring Boot 4, responsável pelas regras de negócio e validações. A persistência de dados é amparada por uma modelagem relacional em PostgreSQL, com comunicação modular via API REST gerenciada por padrões como JPA/Hibernate. Todo o ciclo de desenvolvimento, versionamento de código e cooperação do grupo é centralizado no GitHub através do Git, garantindo o gerenciamento ágil e a qualidade de software exigida na disciplina.

2. METODOLOGIA

A metodologia baseou-se no ciclo de vida de desenvolvimento de sistemas, unindo conceitos teóricos de Engenharia de Software à construção prática da aplicação utilizando os conceitos de Programação vistos em aula. O processo compreendeu etapas sequenciais estruturadas para garantir o rigor técnico, partindo

do levantamento de escopo até a modelagem e a implementação das camadas do software.

A fase inicial consistiu no mapeamento dos requisitos funcionais e fluxos operacionais da loja de roupas para delimitar o comportamento do sistema. Esse diagnóstico guiou o planejamento arquitetural, norteador a escolha de frameworks e padrões de projeto que assegurassem a separação de responsabilidades, o baixo acoplamento e a persistência segura de dados.

A execução do projeto foi conduzida por meio de práticas colaborativas e versionamento contínuo de código. Essa abordagem permitiu a integração organizada dos módulos e a mitigação de inconsistências técnicas. As seções a seguir detalham as decisões e ferramentas adotadas para a viabilização da solução web.

2.1. REQUISITOS FUNCIONAIS

No núcleo do sistema, encontram-se os módulos voltados aos cadastros de base essenciais para o funcionamento do estabelecimento. O sistema permite o gerenciamento completo do cadastro de produtos, o qual armazena de forma detalhada o código, nome, tamanho, cor, tipo, preços de custo e de venda, fornecedor associado, descrição e a respectiva quantidade em estoque. Paralelamente, a aplicação viabiliza o registro de clientes e fornecedores, concentrando dados como CPF/CNPJ, e-mail, telefone e endereço completo. Há também uma área destinada à manutenção dos dados dos próprios funcionários e das configurações da loja, exigindo identificações formais e credenciais de acesso seguras para fins de autenticação.

No âmbito das operações comerciais, o Sistema Vestock centraliza o registro de vendas, permitindo que o funcionário vincule em uma única transação o cliente atendido, as peças de vestuário selecionadas, suas respectivas quantidades, valores aplicados e a forma de pagamento escolhida. Para dar suporte às estratégias de marketing do estabelecimento, a plataforma conta com uma funcionalidade dedicada ao controle de cupons de desconto, os quais podem ser aplicados diretamente nas vendas desde que estejam vinculados a campanhas promocionais vigentes.

Um dos grandes diferenciais operacionais integrados à plataforma é o gerenciamento de vendas na modalidade condicional. Esse recurso permite a saída

temporária de itens de vestuário da loja, registrando formalmente as datas de retirada e as previsões de devolução. O sistema realiza o controle estrito do retorno dessas peças condicionais, garantindo que o fluxo de trânsito das mercadorias seja monitorado do início ao fim e mitigando perdas financeiras.

Por fim, a camada de lógica de negócios do *back-end* atua diretamente sobre o comportamento físico do inventário por meio de automações. Toda vez que uma venda tradicional é consolidada na interface, ou quando ocorre a confirmação de baixa em itens retirados por condicional, o sistema executa de forma imediata e automática a atualização do estoque. Esse gatilho de sincronização garante que a quantidade real de produtos reflita exatamente o estado físico das araras e depósitos da loja, impedindo erros humanos de contagem ou vendas em duplicidade.

2.2. ARQUITETURA E BACKEND

A engenharia do *back-end* do Sistema Vestock foi projetada com foco em manutenibilidade, escalabilidade e forte isolamento de responsabilidades, transpondo os conceitos teóricos de arquitetura de software discutidos em aula para uma implementação prática e robusta. O sistema adota o padrão de Arquitetura em Camadas (*Layered Architecture*), segregando explicitamente os componentes responsáveis pela recepção de requisições, processamento das regras de negócio e persistência de dados. Essa separação garante que modificações na interface ou nas regras de estoque não impactem diretamente o armazenamento dos dados, cumprindo o princípio de responsabilidade única.

O ecossistema de desenvolvimento escolhido baseia-se na linguagem Java 21, combinado com o framework Spring Boot 4. A escolha do Java 21 justifica-se pela utilização de recursos modernos de concorrência e estabilidade da plataforma, enquanto o Spring Boot 4 atua como o acelerador do desenvolvimento através do seu mecanismo de injeção de dependências e configuração automatizada.

Na estrutura interna do projeto, a divisão de responsabilidades inicia-se na camada de controladores, que atua como a porta de entrada da aplicação ao interceptar as requisições HTTP enviadas pela interface e fornecer os retornos padronizados. No contexto do Sistema Vestock, esta camada expõe as rotas da API REST que os funcionários utilizam para acionar comandos no estoque ou listar transações de vendas.

Toda a inteligência do sistema é concentrada na camada de negócio, local onde se executam as validações cruciais da loja de roupas, tais como a verificação de disponibilidade de peças antes da concretização de uma venda, o cálculo de baixa automatizada no inventário e as restrições de acesso baseadas no perfil de funcionário.

Por fim, a camada de persistência é a responsável pela comunicação direta com o banco de dados através da especificação JPA e do provedor Hibernate, que são encapsulados pelo Spring Data JPA, eliminando a necessidade de escrita manual de consultas SQL complexas para operações rotineiras e mapeando as entidades de software diretamente em tabelas relacionais.

2.3. DESIGN PATTERNS

Para solucionar problemas clássicos de acoplamento, estruturar o fluxo de dados e cumprir os requisitos obrigatórios da disciplina, foram incorporados três padrões de projeto fundamentais à arquitetura do servidor, sendo eles Service Layer, Repository Pattern e Dependency Injection.

Service Layer (Camada de Serviço) foi aplicado para isolar completamente as regras de negócio das camadas de apresentação e persistência. No Sistema Vestock, as classes de serviço coordenam as operações do sistema da loja de roupas, executando validações cruciais como a verificação de disponibilidade de peças no inventário antes de concretizar uma venda e o cálculo de baixa automatizada de produtos.

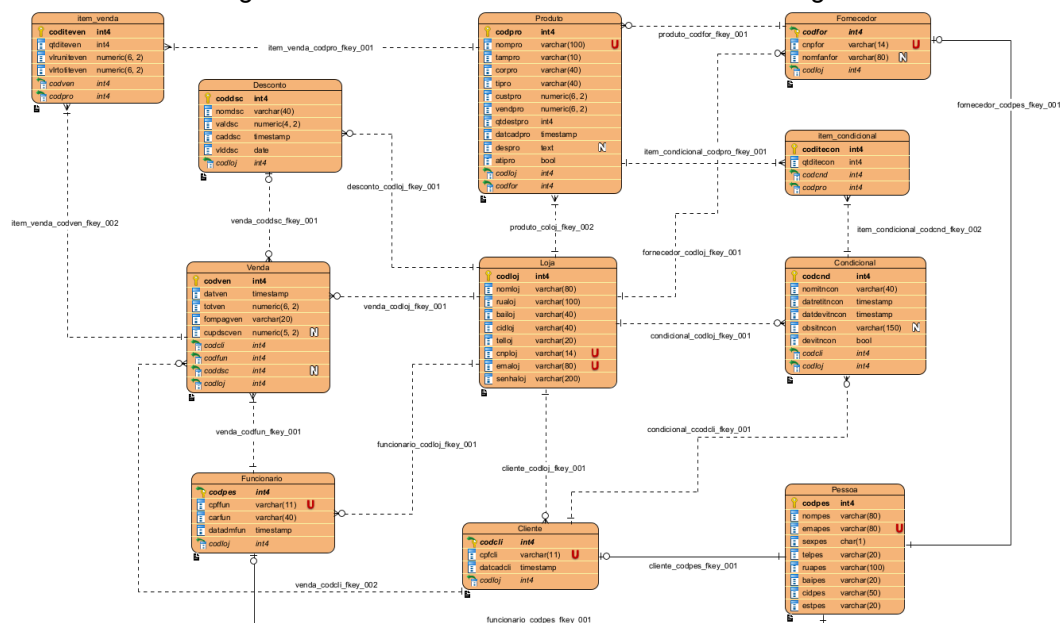
Repository Pattern é utilizado para separar a lógica de persistência de dados da lógica de negócio. Por meio das interfaces do Spring Data JPA, o sistema encapsula o acesso ao banco de dados, permitindo que a camada de serviço manipule os dados de vendas e estoque através de métodos abstratos de alto nível, eliminando a necessidade de declarações SQL manuais no corpo do código.

Dependency Injection é um padrão estrutural gerenciado nativamente pelo *framework* Spring Boot. A inversão de controle ocorre ao injetar automaticamente as instâncias de serviços nos controladores e os repositórios nos serviços. Essa abordagem garante o baixo acoplamento entre os componentes do sistema, elimina a necessidade de instanciação manual via operador *new* e viabiliza a criação de testes unitários isolados de forma eficiente.

2.4. BANCO DE DADOS

A camada de persistência do Sistema Vestock foi construída utilizando o Sistema Gerenciador de Banco de Dados relacional PostgreSQL, gerenciado e administrado com o suporte das ferramentas DBeaver e pgAdmin, cuja escolha fundamenta-se na necessidade de garantir a consistência estrita dos dados estruturados, o suporte a transações ACID e a integridade referencial nas operações de controle de estoque e registros de cupons e vendas. Para a concepção e modelagem visual de toda essa estrutura relacional, utilizou-se o software Visual Paradigm, viabilizando a criação do diagrama de entidade-relacionamento que norteia a arquitetura física dos dados.

Diagrama do Banco de dados feito no Visual Paradigm.



Fonte: Elaborado pelos autores.

O modelo lógico adota o conceito de generalização e especialização na gestão de atores. A tabela abstrata Pessoa centraliza os atributos comuns de identificação e localização como nome, e-mail, sexo, telefone e endereço, enquanto Cliente e Funcionario atuam como especializações que herdaram sua chave primária para evitar a duplicidade de dados.

Os demais componentes estruturais dividem-se em três núcleos integrados para suportar as regras de negócio da loja de roupas. O primeiro é o núcleo de estoque, composto pela tabela Produto que armazena tamanho, cor, tipo, preços e quantidade, estando conectado diretamente à tabela Fornecedor para mapear a

origem das mercadorias. O segundo é o núcleo comercial, estruturado a partir da tabela Venda que é interligada à tabela Desconto para o controle de cupons promocionais.

A tabela Venda se relaciona de forma muitos para muitos com as mercadorias por meio da tabela associativa item_venda para registrar quantidades e valores históricos de cada transação. O terceiro é o núcleo de condicionais, que gerencia retiradas provisórias de roupas por meio das tabelas Condicional e item_condicional, controlando os clientes, os itens retirados e o status de devolução por meio de um campo booleano.

Por fim, toda a arquitetura é centralizada em meio à entidade Loja. Esta chave atua como um delimitador global nas tabelas operacionais, garantindo o isolamento dos dados do estabelecimento e permitindo uma futura expansão para o suporte de múltiplas filiais.

2.5. FRONT-END

A interface do Sistema Vestock foi desenvolvida utilizando o framework Flutter em conjunto com o Dart SDK 3.10.1, viabilizando a criação de uma aplicação multiplataforma a partir de um único código-fonte, garantindo suporte nativo para ambiente web, dispositivos móveis e sistemas desktop. A arquitetura do *front-end* foca em criar uma experiência de usuário intuitiva e responsiva para o público interno da loja de roupas, adaptando os layouts dinamicamente de acordo com o tamanho da tela do dispositivo utilizado pelo funcionário.

A integração com o servidor ocorre por meio de uma API REST, que funciona como a camada intermediária de comunicação do ecossistema, onde o cliente em Flutter realiza requisições assíncronas mediadas pelo pacote http para consumir os pontos de extremidade expostos pelo *back-end* em Java. Essa API trafega os dados estruturados em formato JSON, permitindo que as operações de criação, leitura, atualização e exclusão de cadastros e movimentações de estoque sejam processadas e renderizadas em tempo real na tela do usuário com validação imediata de status.

Para garantir a eficiência operacional e a persistência de estados locais no dispositivo, a aplicação utiliza o pacote shared_preferences, responsável por armazenar pequenas coleções de dados de forma leve, como tokens de

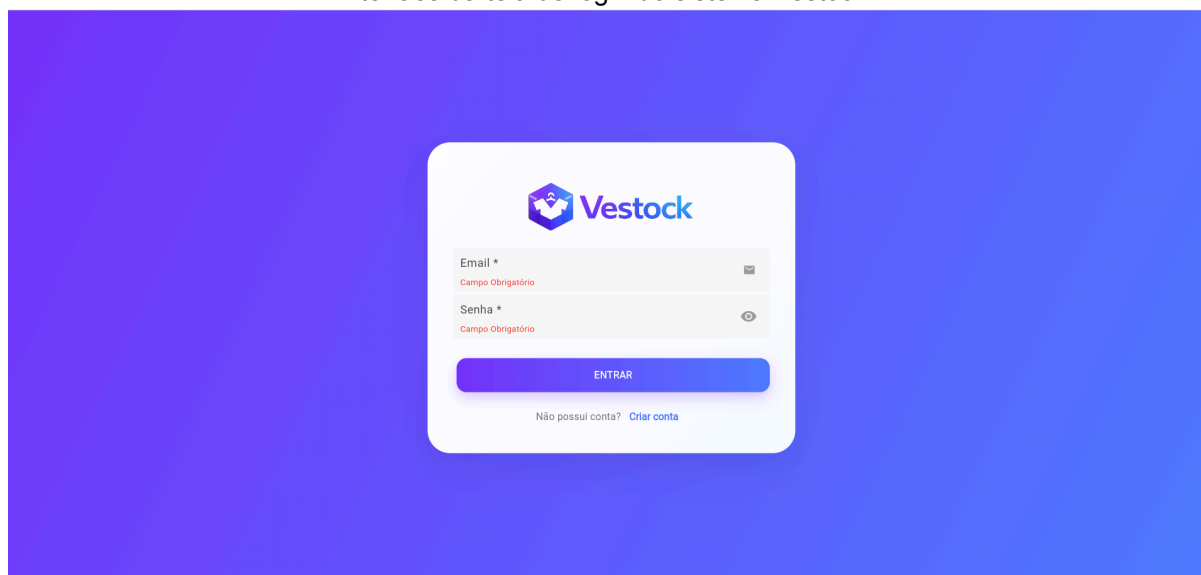
autenticação da sessão do funcionário e preferências de configuração da loja de roupas.

O tratamento visual e a formatação de dados complexos, como a exibição de moedas e a padronização cronológica de datas de cadastro ou devolução de condicionais, são gerenciados pelo pacote intl, assegurando a conformidade com as regras de localização do sistema. A usabilidade e o dinamismo da interface são potencializados pela biblioteca flutter_slidable, que introduz componentes visuais deslizantes para ações rápidas de exclusão ou edição de itens, e pelo pacote awidgets, que fornece componentes customizados e padronizados para acelerar a construção de formulários de vendas, listagens de produtos e cadastros base com alta fidelidade visual.

3. RESULTADOS

As interfaces iniciais do Sistema Vestock compreendem as telas de autenticação e de registro da organização, projetadas com foco no minimalismo e na clareza visual para o usuário. A tela de login apresenta um formulário centralizado composto pelos campos de e-mail e senha, ambos acompanhados de validadores estritos em tempo real que indicam campos obrigatórios vazios antes mesmo do envio da requisição ao servidor.

Interface da tela de login do sistema Vestock.

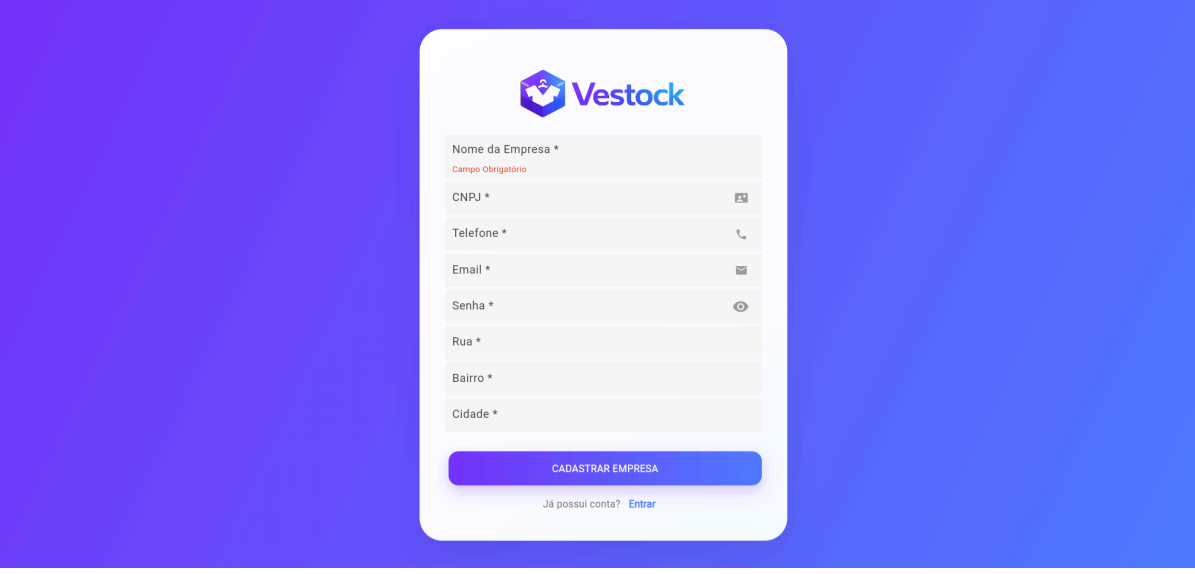


Fonte: Elaborado pelos autores.

Abaixo do botão de acesso principal, há um link direcionador para a tela de criação de conta, que por sua vez expande o escopo de entrada de dados para o

cadastro completo do estabelecimento. Para realizar o cadastro, esta segunda interface exige o preenchimento formal de metadados corporativos e de localização, incluindo a razão social ou nome da empresa, CNPJ, telefone de contato, credenciais de acesso administrativas e o endereço composto por rua, bairro e cidade.

Interface da tela de cadastro de empresa do sistema Vestock.



Nome da Empresa *

Campo Obrigatório

CNPJ *

Telefone *

Email *

Senha *

Rua *

Bairro *

Cidade *

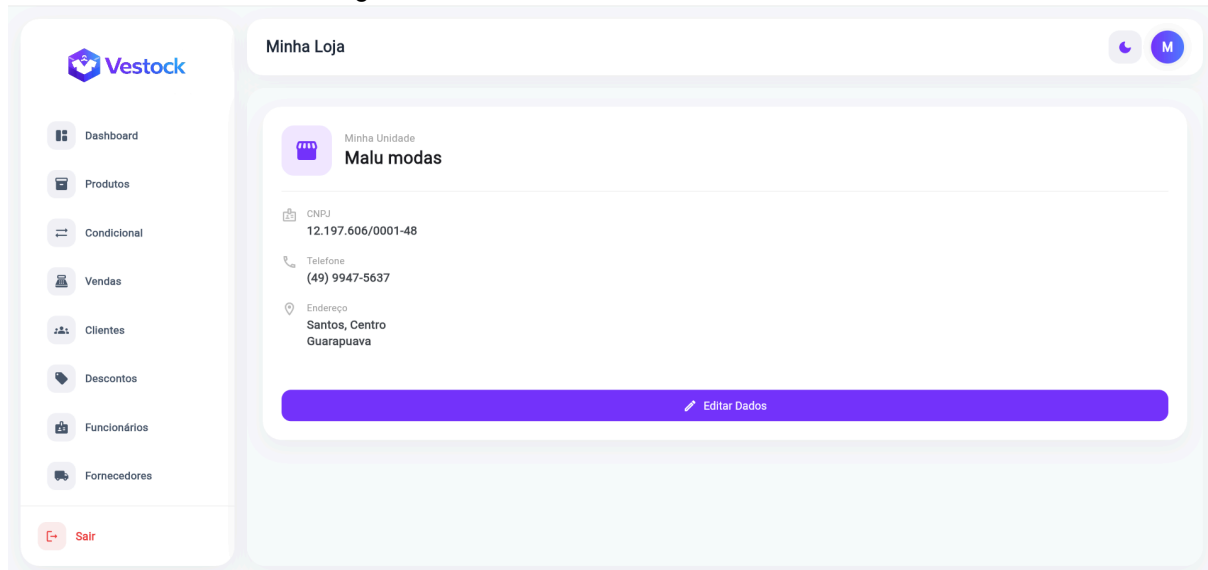
CADASTRAR EMPRESA

Já possui conta? [Entrar](#)

Fonte: Elaborado pelos autores.

Após a validação bem-sucedida das credenciais de acesso, o usuário é direcionado para a tela de gerenciamento institucional intitulada Minha Loja. Esta interface funciona como o cartão de visitas operacional do estabelecimento, exibindo de forma clara o nome da unidade cadastrada, o CNPJ correspondente, o telefone de contato e o endereço completo com rua, bairro e cidade. Um botão centralizado de edição de dados permite a atualização ágil dessas informações institucionais diretamente no banco de dados.

Página dos dados da conta do sistema Vestock.

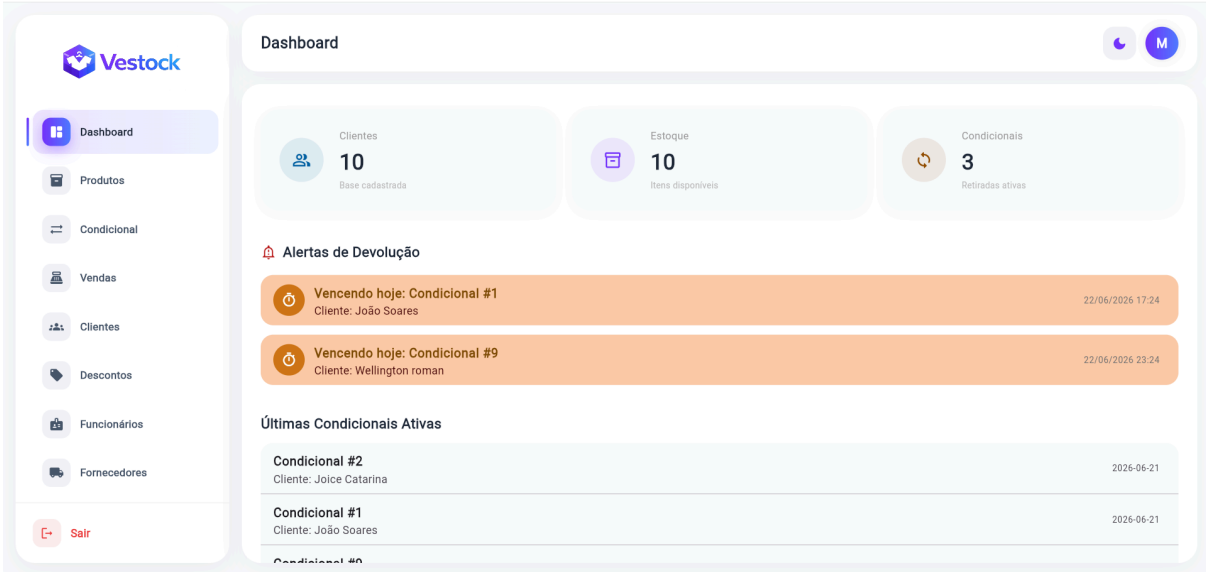


Fonte: Elaborado pelos autores.

Toda a navegação é mediada por um menu lateral esquerdo que centraliza o acesso aos recursos do sistema, permitindo selecionar as abas de Produtos, para catalogação e estoque, Condicional, para rastreamento de retiradas provisórias, Vendas, para registro de transações, Clientes e Fornecedores, para dados cadastrais, Descontos, para gestão de cupons, e Funcionários, para gestão dos funcionários cadastrados. Por fim, a opção “Sair” destacada em vermelho para o encerramento seguro da seção.

Ao selecionar a opção Dashboard, a interface carrega o painel de indicadores estratégicos em tempo real para apoio gerencial. A área superior exibe cartões com resumos quantitativos de clientes, estoque e condicionais ativos, enquanto a seção inferior destaca alertas visuais de devoluções que vencem no dia atual e uma listagem cronológica das últimas movimentações registradas na plataforma.

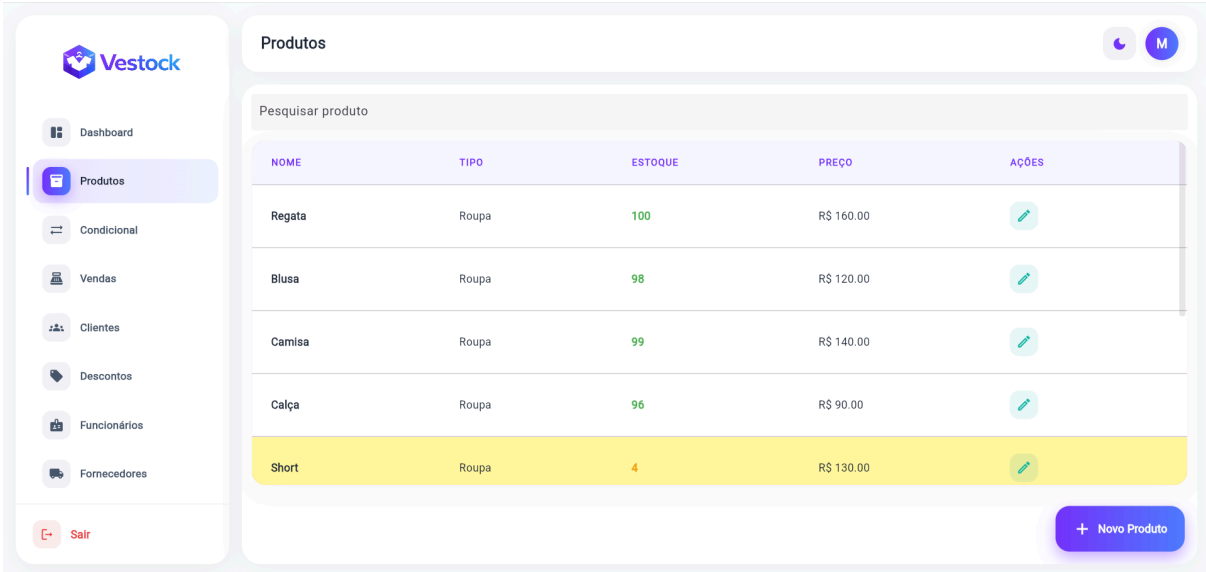
Aba Dashboard do sistema Vestock.



Fonte: Elaborado pelos autores.

A aba de Produtos permite o cadastro e a catalogação de peças de roupas, o controle de estoque e o gerenciamento de variações como tamanhos e cores.

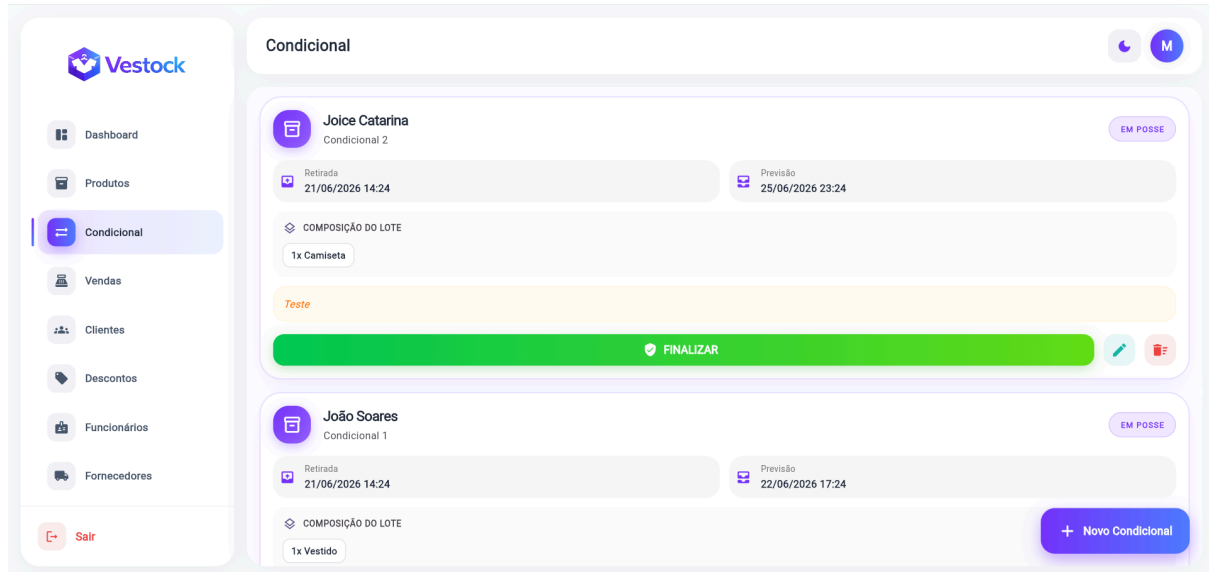
Aba Produtos do sistema Vestock.



Fonte: Elaborado pelos autores.

No módulo de Condicional, realiza-se o registro e o rastreamento das retiradas provisórias de mercadorias por clientes, incluindo a validação do status de devolução.

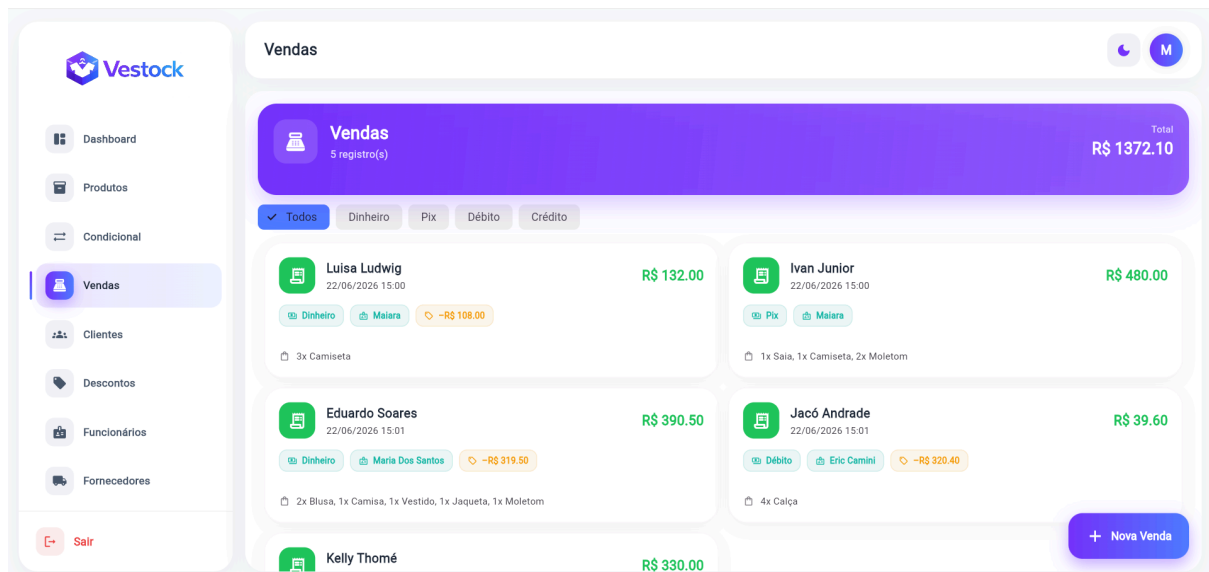
Aba Condicional do sistema Vestock.



Fonte: Elaborado pelos autores.

A seção de Vendas é destinada à abertura, ao registro e à baixa de transações comerciais em tempo real.

Aba Vendas do sistema Vestock.



Fonte: Elaborado pelos autores.

O gerenciamento de públicos e parceiros comerciais é distribuído entre as abas de Clientes e Fornecedores, que organizam os dados cadastrais, históricos e informações de contato de cada entidade.

Aba Clientes do sistema Vestock.

Dashboard

Produtos

Condicional

Vendas

Clientes

Descontos

Funcionários

Fornecedores

Sair

Clientes

Pesquisar clientes...

NOME	CPF	TELEFONE	CIDADE	AÇÕES
Joice Catarina	784.051.888-51	(49) 9999-91004	Guarapuava	
João Soares	687.924.696-63	(49) 9999-91003	Guarapuava	
Jacó Andrade	092.389.408-00	(49) 9999-91005	Buriti	
Emily Andrade	237.060.858-79	(49) 9999-91006	São Miguel do Oeste	
Juan Junior	713.139.367-31	(49) 9999-91007	Itanirama	

+ Novo Cliente

Fonte: Elaborado pelos autores.

Aba Fornecedores do sistema Vestock.

Dashboard

Produtos

Condicional

Vendas

Clientes

Descontos

Funcionários

Fornecedores

Sair

Fornecedores

Textil Ltda

87.473.993/7373-45

textil@gmail.com

Curitiba

Roupas

18.631.579/0001-56

pessoa16@teste.com

Joaçaba

Malwee

70.599.191/0001-35

pessoa13@teste.com

São Paulo

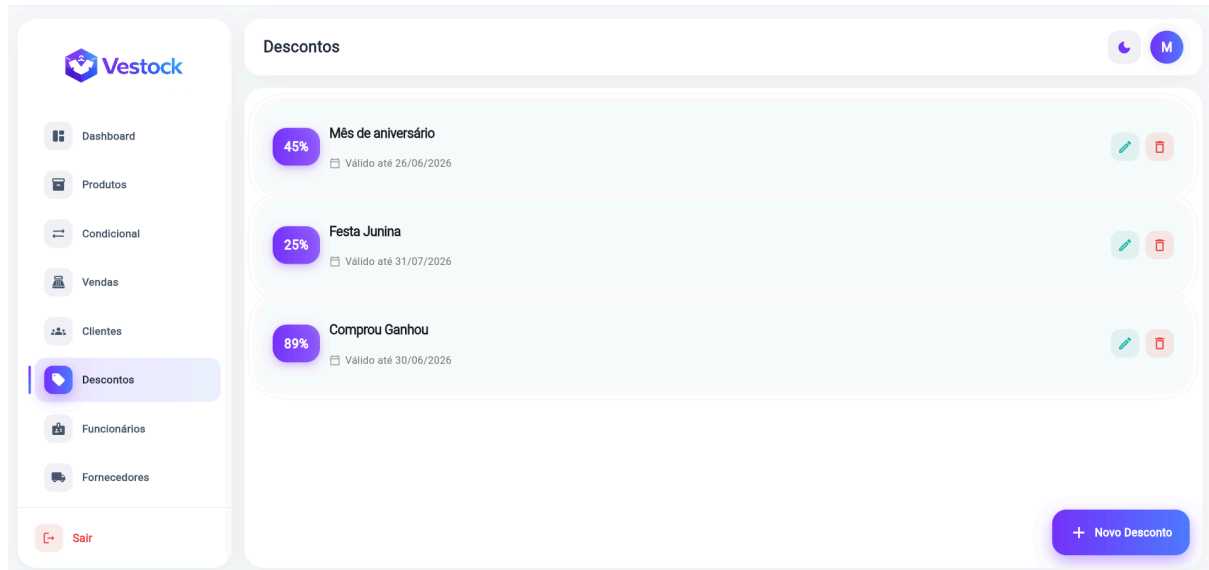
Joice Textil

+ Novo Fornecedor

Fonte: Elaborado pelos autores.

O controle promocional ocorre na aba de Descontos, responsável pelo gerenciamento de cupons e campanhas ativas na loja de roupas.

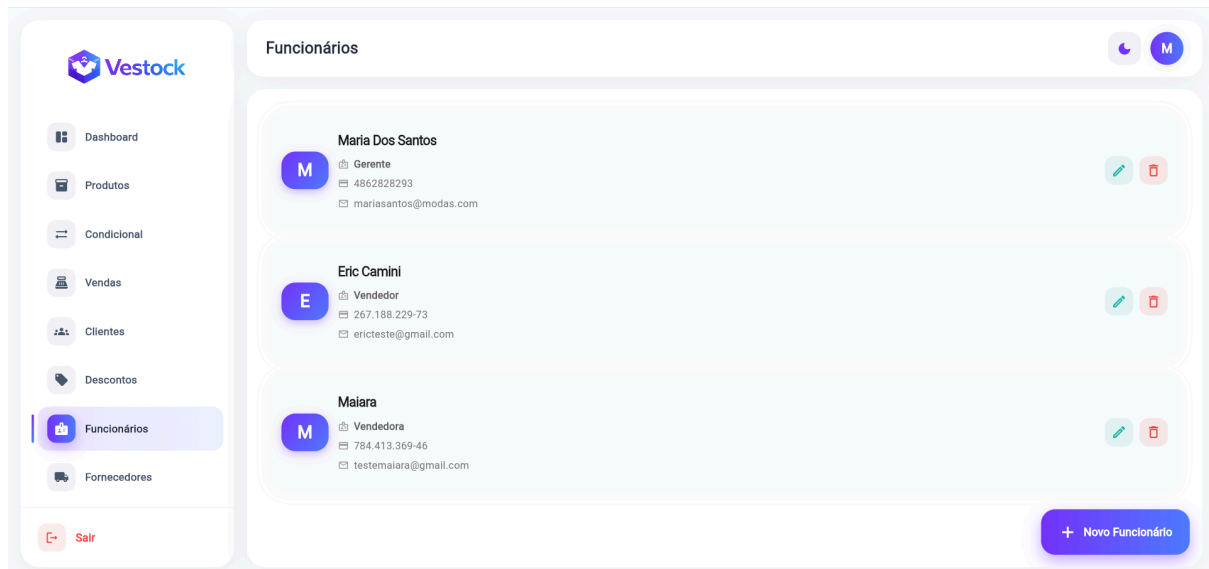
Aba Descontos do sistema Vestock.



Fonte: Elaborado pelos autores.

A segurança e a hierarquia do sistema são coordenadas na aba de Funcionários, utilizada para cadastrar os operadores internos e configurar seus respectivos perfis de acesso, complementando o menu com um botão destacado em vermelho para o encerramento seguro da sessão de uso.

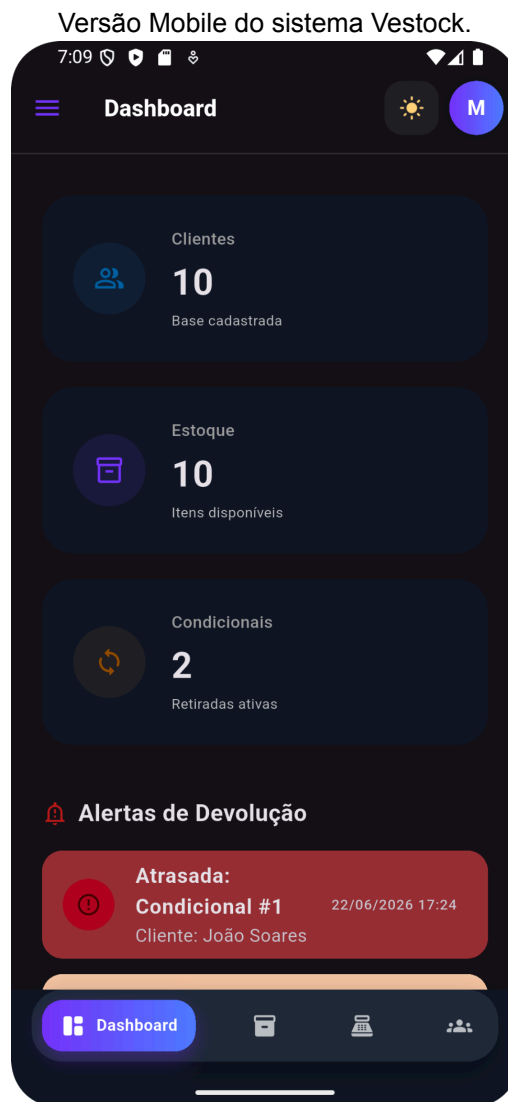
Aba Funcionários do sistema Vestock.



Fonte: Elaborado pelos autores.

A versão mobile do Sistema Vestok adapta os recursos da plataforma para dispositivos portáteis utilizando uma interface otimizada para o uso dinâmico dos funcionários da loja. O painel Dashboard nessa versão redistribui os cartões informativos verticalmente para se ajustar às telas menores, exibindo de forma clara

os indicadores de clientes cadastrados, itens disponíveis no estoque e o volume de condicionais ativos. Abaixo das métricas, destaca-se a seção de alertas de devolução, que exibe notificações com sinalização visual em vermelho para apontar status críticos, como o atraso na entrega de mercadorias por clientes específicos.



Fonte: Elaborado pelos autores.

A navegação principal deixa o formato lateral e passa a ser gerenciada por uma barra de navegação inferior persistente. Por meio dela, o operador pode selecionar de maneira ágil as abas mais utilizadas no cotidiano da loja de roupas, incluindo o próprio Dashboard, o estoque de Produtos, a abertura de Vendas e o cadastro de Clientes. Além disso, a interface apresenta a ativação do modo escuro (*dark mode*), demonstrando o foco no conforto visual do usuário interno durante longas jornadas de trabalho e a flexibilidade do design responsivo do ecossistema.